

Vendredi 27/02/2026

Rapport Back- end Partie : messaging

__init__.py : ce fichier permet de déclarer le dossier messaging comme un package Python. Même s'il est vide, il est nécessaire pour que Django puisse reconnaître et importer correctement les modules de l'application.

Concrètement, il permet :

- D'écrire : `from app.messaging.models import Message`
- De structurer proprement le projet Django.
- À Django de reconnaître l'application comme un module valide.

models.py : Ce fichier ne crée aucun modèle. Il importe simplement les modèles Message et Conversation depuis l'application notifications pour les réutiliser dans l'application messaging.

- Il évite la duplication des modèles.
- Il empêche les conflits dans la base de données.
- Il centralise la logique des modèles dans une seule application.

Utilité : garder une architecture propre et organisée.

admin.py : Ce fichier configure l'interface d'administration Django pour gérer les messages et conversations, Il permet :

- D'afficher les messages avec leurs informations principales.
- De voir un aperçu du texte.
- De filtrer par date ou statut (lu/non lu).
- D'afficher les participants d'une conversation.

- De suivre la dernière activité.

Utilité : permettre à l'administrateur de contrôler la messagerie depuis le panneau admin.

permissions.py : Ce fichier définit des règles de sécurité pour l'API.

Il contient deux permissions :

- L'utilisateur doit être expéditeur ou destinataire d'un message.
- L'utilisateur doit faire partie d'une conversation pour y accéder.

Utilité : protéger les données et empêcher l'accès non autorisé.

serializers.py : Ce fichier transforme les objets de la base de données en format JSON pour les envoyer au frontend, Il gère :

- Les informations minimales d'un utilisateur.
- La transformation des messages en données JSON.
- La transformation des conversations.
- La construction des URLs complètes pour les photos et médias.
- Le calcul du nombre de messages non lus.
- L'ajout des informations du trajet dans les conversations de groupe.

Utilité : assurer la communication structurée entre backend et frontend.

views.py : Ce fichier contient la logique principale de l'API REST pour la messagerie, Il permet :

- **Gestion des messages :**

- Créer un message.
- Lister les messages d'un utilisateur.
- Récupérer une conversation privée.

- Récupérer les messages d'un groupe lié à un trajet.
- Compter les messages non lus.
- Marquer un message comme lu.
- Marquer tous les messages comme lus.
- Envoyer des fichiers (images, médias) en base64.
- Mettre à jour automatiquement la conversation après un message.

- Gestion des conversations :

- Lister les conversations de l'utilisateur.
- Lister ses groupes.
- Récupérer les messages d'une conversation.

Utilité :

- C'est le cœur du système API.
- Il relie la base de données, les serializers et les permissions.
- Il prépare aussi l'URL WebSocket pour la messagerie temps réel.

consumers.py : Ce fichier gère la messagerie en temps réel avec WebSocket via Django Channels, Il permet :

- La connexion WebSocket.
- La vérification que l'utilisateur est authentifié.
- La vérification des permissions.
- L'envoi instantané des messages.
- L'indicateur "en train d'écrire".
- L'envoi de l'historique des messages.
- La sauvegarde des messages en base de données.

- La mise à jour de la conversation.

Il gère deux types :

- Conversation de groupe (trajet).
- Conversation privée (entre deux utilisateurs).

Utilité : permettre un chat instantané sécurisé comme WhatsApp.

middleware.py : Ce fichier crée un middleware personnalisé pour authentifier les connexions WebSocket avec JWT, Il fait :

- Récupérer le token depuis l'URL.
- Vérifier le token.
- Identifier l'utilisateur.
- Ajouter l'utilisateur dans scope ["user"].
- Refuser l'accès si le token est invalide.

Utilité : sécuriser la messagerie en temps réel.

routing.py : Ce fichier définit les routes WebSocket, Il associe :

- URL chat groupe → ChatConsumer
- URL chat privé → ChatConsumer
- URL notifications → NotificationConsumer

Utilité : connecter les bonnes URLs aux bons consommateurs WebSocket.

urls.py : Ce fichier définit les routes API REST pour la messagerie, Il génère automatiquement :

- Liste des messages.
- Création de message.
- Détail d'un message.
- Liste des conversations.
- Upload de média.

Utilité : exposer les endpoints API pour le frontend.

Oudjane_Anias_2CP7